

UrbanCart Analytics Term Project

A Complete Data Pipeline: SQL Extraction, Data Cleaning, Statistical Analysis, and Business Recommendations

Prepared for UrbanCart Leadership
Big Data Term Project — Phnom Penh International University

Table of Contents

Executive Summary

UrbanCart's transaction history (9,000 orders, 20,176 order lines, 2,500 customers, 2022-01-02 through 2024-12-31) supports five headline findings for leadership.

Top 5 Findings

- 1. Revenue is concentrated in two categories. Books & Media (\$3.84M) and Beauty & Health (\$3.04M) generate nearly half of total revenue despite similar order counts to lower-performing categories — the driver is average order value, not traffic.
- 2. A small "Champions" customer segment drives outsized value. RFM segmentation identifies 472 Champions (of 2500) generating the largest share of lifetime revenue, with an average age of ~45.
- 3. Three products are essentially guaranteed to stock out. Monte Carlo simulation (10,000 trials) on current inventory vs. demand puts products 300, 222, and 105 at ~100% probability of stockout within the 14-day supplier lead time.
- 4. Data-quality issues are real but manageable, and matter to the bottom line. Naively ignoring return line-items overstates revenue by 2.97% (~\$396K) — a difference large enough to change category-profitability conclusions if missed.
- 5. The legacy CRM export is salvageable but needs standardized cleaning. Of 1,428 raw legacy rows, 1,345 survive as clean, de-duplicated, date-standardized records — a 5.8% reduction, mostly from duplicate detection rather than data loss.

Top Recommendations

- Immediately place reorder requests for products 300, 222, and 105 (recommended reorder points ~22-24 units each).
- Build a retention program targeted at the 434 "At Risk" and 567 "Churned" segment customers before they fully lapse.
- Standardize signup-date capture across marketing systems to avoid repeating the 4-format inconsistency found in the legacy export.
- Investigate why Electronics and Apparel underperform Books & Media and Beauty & Health on AOV despite comparable order volume.

1. Introduction & Business Context

UrbanCart is a mid-size online retailer selling across six product categories — Electronics, Home & Kitchen, Apparel, Sports & Outdoors, Beauty & Health, and Books & Media — to customers spread across fifteen cities worldwide. Leadership commissioned this analysis to move from raw operational data to decision-ready answers on three questions:

- Who are UrbanCart's most valuable customers, and what drives their loyalty or churn?
- Which products and categories are actually profitable once returns, discounts, and cost are accounted for — and can near-term demand be forecast?
- Is the marketing/signup data, spread across an old CRM export and the current database, trustworthy enough to make decisions on?

These questions matter because UrbanCart is making inventory, marketing-spend, and retention decisions today using data that has never been systematically reconciled or audited. Getting the answers wrong has direct financial consequences: an inflated revenue figure from ignoring returns, for instance, could lead to overstating a category's health and misallocating marketing budget toward it.

This report documents the full pipeline used to answer these questions — SQL extraction against the operational database, pandas-based cleaning and integration of two messy external files, four NumPy-implemented statistical routines, and the resulting business findings — and treats the dataset as "real": it contains genuine data-entry errors, duplicates, and inconsistencies, and part of the analytical job was noticing and correctly handling them rather than assuming clean inputs.

The report is organized to mirror the pipeline itself. Section 2 describes the data as received, before any cleaning, so the scale of the problem is visible. Section 3 walks through the SQL extraction layer, since every later stage of analysis ultimately depends on data pulled through those ten queries. Section 4 is the most detailed section by design — the project brief's page budget allocates more space to data cleaning than to any other section, reflecting a genuine truth about applied analytics work: the two external files alone required standardizing four different date formats, resolving 80 duplicate or near-duplicate customer records, and reconciling a differently-named, partially-overlapping supplier catalog, none of which is visible in a polished final chart but all of which determines whether that chart is trustworthy. Section 5 documents the four required NumPy routines with enough mathematical detail that a reader without access to the code could still verify the logic by hand on a small example. Sections 6 through 8 turn the cleaned data and statistical output into the eight specific business answers leadership asked for, followed by an honest accounting of where those answers could be wrong and what to do next.

1.1 Pipeline Overview

Concretely, the analysis is implemented as three sequential Python stages plus the SQL extraction layer that feeds them, matching the four phases in the project brief:

- Phase 1 (SQL): ten queries run directly against ecommerce.db answer the specific extraction questions in Section 3 and also load four query results into pandas for reuse downstream.
- Phase 2 (pandas): phase2_cleaning.py takes the raw database tables plus the two external CSVs and produces ten cleaned, standardized files in data/processed/ — this is the stage documented in Section 4.
- Phase 3 (NumPy): phase3_numpy.py reads only the Phase 2 outputs and implements the four required statistical routines (RFM, cosine similarity, normal-equation regression, Monte Carlo simulation) using raw NumPy arrays rather than convenience library calls — documented in Section 5.
- Phase 4 (visualization): phase4_visuals.py reads the Phase 2 and Phase 3 outputs and produces the eight charts answering the specific business questions in Section 6, plus a machine-readable findings summary.

Because each stage only reads the previous stage's saved output files rather than re-deriving anything from raw sources, the whole pipeline is re-runnable end to end with three commands (see Appendix C), and any single stage can be re-run in isolation once its upstream dependency is up to date — useful if, for example, only the visualization styling needs to change without re-running the full cleaning and statistical pipeline.

2. Data Description

Data comes from two source types: a SQLite operational database (ecommerce.db) with six tables, and two external CSV exports that were never integrated into the database.

2.1 Database Schema

Table	Rows	Key Columns	Known Issues
customers	2500	customer_id (PK), email, signup_date, city, age, gender	63 missing city, 139 missing age, 115 missing gender
products	300	product_id (PK), category, unit_price, cost	3 extreme price outliers
orders	9000	order_id (PK), customer_id (FK), order_date, status	status: completed / returned / cancelled / pending
order_items	20,362 raw	order_item_id (PK), order_id (FK), product_id (FK), quantity, discount	186 exact-duplicate rows; negative quantity = return
reviews	4,000 raw	review_id (PK), product_id (FK), customer_id (FK), rating	41 out-of-range ratings; 828 missing text (20.9%)
web_sessions	12000	session_id (PK), customer_id (FK), device, duration_minutes	no major issues found

2.2 External Files

legacy_customers_export.csv — an old marketing-system export with 1,428 rows, four inconsistent date formats, inconsistent name casing, a header with trailing whitespace, duplicate/near-duplicate customers, missing emails, and junk test-account rows.

product_catalog_2024.csv — a supplier-provided catalog with different column names/casing than the products table, partial coverage overlap (255 of 300 products), and 12 supplier-only SKUs absent from the database entirely.

2.3 Scale & Coverage

The dataset spans 2022-01-02 to 2024-12-31 (36 months), covering 2,500 customers across 9 countries and 16 cities, 300 products across 6 categories, 9,000 orders, and 12,000 web sessions.

2.4 Known Limitations

- Product names in the catalog appear templated/synthetic (e.g. "White Fitnes", "Improve Smartphone") rather than real product naming — consistent with this being a generated teaching dataset. Category and price fields are still treated as reliable.
- The database has no explicit customer-acquisition-channel field, limiting root-cause analysis of why some cities show higher revenue-per-customer than others.
- Order status (completed/returned/cancelled/pending) is not fully cross-validated against order_items' negative-quantity return flag in this analysis; the two could disagree in edge cases and were treated as two independent signals rather than reconciled.

2.5 Why This Matters for the Three Business Questions

Each data-quality issue documented above has a direct line to one of the three business questions from Section 1. The customer-table gaps (missing city, age, gender) bear on Question 1 (who are the most valuable customers) because RFM segmentation and any demographic breakdown of that segmentation depend on those fields being handled consistently rather than silently dropped or guessed. The product and order_items issues (price outliers, duplicate lines, negative quantities) bear directly on Question 2 (which products are actually profitable) because

every profitability number in this report is a function of unit_price, cost, discount, and quantity — get any one of those wrong at scale and every downstream margin calculation inherits the error. The legacy CSV's inconsistencies bear on Question 3 (is the marketing/signup data trustworthy) most directly, since that file is UrbanCart's only historical view into pre-database signup activity, and its quality directly determines whether leadership can trust any trend built from it.

3. SQL Extraction & Methodology

All ten required queries were written against ecommerce.db and validated by direct execution (see Appendix A for full SQL text; this section summarizes each query's purpose and result).

Query 1 — Revenue, orders & AOV by category

Total revenue, order count, and average order value per product category, net of discounts.

category	order_count	total_revenue	avg_order_value
Books & Media	2861	3,839,920.5	1,342.16
Beauty & Health	2746	3,044,659.32	1,108.76
Apparel	2782	1,756,735.57	631.46
Sports & Outdoors	2760	1,698,958.48	615.56
Electronics	2819	1,694,042.34	600.94
Home & Kitchen	2864	1,443,477.26	504.01

(showing first 6 rows of the query result)

Books & Media leads at \$3.84M revenue across 2,861 orders (AOV \$1,342); Beauty & Health follows at \$3.04M. Electronics, Apparel and Sports & Outdoors cluster much lower (\$1.6M-\$1.7M) despite similar order counts — average order value, not order volume, is what separates the top two categories.

Query 2 — Top 20 customers by lifetime spend

Highest lifetime-spend customers via a three-way JOIN across customers, orders, and order_items.

customer_id	name	city	signup_date	lifetime_spend
611	Tara Holmes	Sydney	2021-10-15	146,989.77
689	Misty Mcguire	Toronto	2020-08-25	108,395.86
2181	Jason Ramirez	Melbourne	2019-11-08	106,449.54
1259	Matthew Perez	Los Angeles	2023-02-25	95,535.58
1773	Connie Hamilton	Los Angeles	2022-02-13	80,690.04
1217	Virginia Hernandez	Paris	2022-07-07	79,992.06
528	Brent Hopkins	Vancouver	2023-10-12	74,392.75
2445	Virginia Rose	Bangkok	2023-11-26	72,592.96

(showing first 8 rows of the query result)

The single highest-spend customer (Tara Holmes, Sydney) totals \$146,990 lifetime — more than the next customer's spend by 35%. The top 20 skew toward Sydney, Toronto, Melbourne and Los Angeles.

Query 3 — Month-over-month revenue trend

24-month revenue trend using LAG() for month-over-month change and SUM() OVER for a running total.

year_month	revenue	prev_month_revenue	mom_change	running_total
2024-12	501,710.04	620,135.92	-118,425.88	13,477,793.46
2024-11	620,135.92	472,820.35	147,315.57	12,976,083.41
2024-10	472,820.35	493,162.67	-20,342.32	12,355,947.49
2024-09	493,162.67	638,991.31	-145,828.63	11,883,127.14
2024-08	638,991.31	544,409.02	94,582.29	11,389,964.46
2024-07	544,409.02	510,424.74	33,984.28	10,750,973.15
2024-06	510,424.74	527,813.96	-17,389.22	10,206,564.14
2024-05	527,813.96	443567	84,246.96	9,696,139.4

(showing first 8 rows of the query result)

Revenue is volatile month to month (swings of +/- \$100K-\$150K are common) but trends upward overall — the running total crossed \$13.4M by December 2024.

Query 4 — Return rate by category

Share of order_item rows with negative quantity (returns), computed per category via a CTE.

category	total_line_items	return_line_items	return_rate
Beauty & Health	3354	106	0.03
Electronics	3401	106	0.03
Apparel	3372	102	0.03
Books & Media	3455	93	0.03
Home & Kitchen	3468	91	0.03
Sports & Outdoors	3312	85	0.03

(showing first 6 rows of the query result)

Return rates cluster tightly between 2.6% (Home & Kitchen) and 3.2% (Beauty & Health) — no category stands out as a return-rate outlier.

Query 5 — Customers active in all of the last 3 quarters

Customers with at least one order in each of Q2, Q3, and Q4 2024, using *HAVING* with *COUNT(DISTINCT quarter)*.

customer_id	quarters_active
2	3
24	3
53	3
63	3
73	3
79	3
89	3
95	3

(showing first 8 rows of the query result)

148 customers ordered in every one of the last three quarters — a useful working definition of "consistently engaged" customers for retention analysis.

Query 6 — Highest-rated products with sufficient review volume

Top 10 products by average rating among products with 15+ reviews (*JOIN* + *GROUP BY* + *HAVING*).

product_id	name	category	review_count	avg_rating
200	White Fitnes	Sports & Outdoors	15	4.6
19	Improve Smartphone	Electronics	15	4.53
191	Yeah Camping	Sports & Outdoors	15	4.53
283	Strategy Children	Books & Media	15	4.53
21	Right Camera	Electronics	16	4.5
66	Wide Cookware	Home & Kitchen	15	4.4
182	Answer Fitnes	Sports & Outdoors	15	4.4
122	Attorney Men	Apparel	17	4.35

(showing first 8 rows of the query result)

The top-rated qualifying product ("White Fitness", Sports & Outdoors) averages 4.6 out of 5 across exactly 15 reviews — right at the review-volume threshold, so this ranking should be treated as provisional until more reviews accumulate.

Query 7 — Session engagement by device (purchasers only)

Average session duration and pages viewed by device type, restricted to customers with at least one order via EXISTS.

device	avg_duration_minutes	avg_pages_viewed	session_count
desktop	6.25	4.7	3877
mobile	6.23	4.64	3845
tablet	6.11	4.74	3970

(showing first 3 rows of the query result)

Desktop, mobile, and tablet sessions are nearly identical in duration (~6.1-6.25 min) and pages viewed (~4.6-4.7) — device choice alone does not meaningfully differentiate engagement depth among buyers.

Query 8 — Product revenue rank within category

RANK() of products by revenue, partitioned by category.

category	product_id	name	revenue	revenue_rank
Apparel	136	Watch Footwear	98,121.07	1
Apparel	107	Appear Women	77,740.15	2
Apparel	131	Spring Kid	75,488.21	3
Apparel	139	About Men	74,895.4	4
Apparel	104	Major Men	74,526.58	5
Apparel	142	Card Footwear	67,679.14	6
Apparel	123	Anything Footwear	64416	7
Apparel	116	Ahead Men	59676	8

(showing first 8 rows of the query result)

Category leaders are highly concentrated: in Apparel, the #1 product ("Watch Footwear", \$98,121) outsells the #2 product by 26%, suggesting a small number of hero SKUs drive a disproportionate share of category revenue.

Query 9 — Payment method mix by country

Share of orders by payment method, split by country.

country	payment_method	order_count	share_of_country_orders
Australia	gift_card	276	0.21
Australia	debit_card	273	0.21
Australia	credit_card	270	0.2
Australia	paypal	257	0.19
Australia	bank_transfer	243	0.18
Canada	gift_card	270	0.22
Canada	credit_card	259	0.21
Canada	paypal	253	0.21

(showing first 8 rows of the query result)

Payment method is close to evenly split across gift card, debit, credit, PayPal, and bank transfer in every country (roughly 18-21% each) — customers are not gravitating strongly toward any one method, so payment-method promotions are unlikely to shift behavior much.

Query 10 (custom) — Revenue per customer by city

Total revenue divided by customer count for cities with at least 20 customers, to find where the customer base is most valuable rather than just largest.

city	n_customers	total_revenue	revenue_per_customer
Sydney	182	1,213,147.06	6,665.64
New York	158	1,052,310.48	6,660.19
Vancouver	172	1,005,443.4	5,845.6
Toronto	143	825,960.11	5,775.94
Los Angeles	149	853,969.2	5,731.34
Singapore	178	971,397.22	5,457.29
Munich	150	796,600.39	5,310.67
Manchester	157	833,663.09	5,309.96

(showing first 8 rows of the query result)

Sydney and New York top revenue-per-customer (~\$6,660 each) despite Sydney having more total customers — this reframes which cities deserve marketing investment versus which just happen to be large.

At least four of these queries (1, 2, 4, and 9) were loaded directly into pandas DataFrames via `sqlite3/pandas.read_sql` rather than pasted as static output, keeping them connected to the Python pipeline for Phases 2-4.

4. Data Cleaning & Integration

Every cleaning decision below follows one principle: flag and document rather than silently overwrite. Nothing was `fillna(0)`d without a stated reason, and every drop/impute decision is logged in `cleaning_log.txt`, generated automatically each pipeline run.

4.1 Legacy CSV: Date Standardization

The legacy export mixed four date formats in the same column: ISO (YYYY-MM-DD), abbreviated-month (DD-Mon-YYYY), long-form (Month DD, YYYY), and US slash (MM/DD/YYYY). A format-trial parser was applied row-by-row, trying each format in turn and falling back to pandas' general parser only if all four failed. All 1,428 non-blank rows parsed successfully into a single datetime dtype — zero unparseable dates.

4.2 Duplicate & Near-Duplicate Customers

Exact duplicates were identified by matching email address (keeping the most recent record); 48 rows were removed this way. A fuzzy pass then caught near-duplicates lacking a reliable email match — same lowercased name plus same city — removing a further 32 rows. One row ("TEST ACCOUNT", `test@test.com`, signup date 1900-01-01) was identified as a junk/test account and dropped outright. Net: 1,428 raw rows -> 1,345 clean rows, a reduction driven almost entirely by legitimate duplicate removal rather than data loss.

4.3 Missing Values: Policy by Field

Field	% Missing	Policy	Rationale
customers.city	3%	Fill with 'Unknown'	Preserves the row's order history; city is descriptive, not used in joins.
customers.age	6%	Leave NaN, flag with <code>age_missing</code>	Imputing a mean age would silently bias any age-based segmentation; better to exclude explicitly.
customers.gender	5%	Fill with 'Not specified'	Not used in any revenue calculation; explicit category avoids dropping otherwise-valid rows.
reviews.review_text	20.9%	Leave NaN, flag with <code>has_text</code>	Rating-based analyses (Q4, Q6) don't need text; imputing text would fabricate opinions.
legacy.email	3.6%	Leave NaN, retained	Row still has usable name/city/date; email is only needed for exact-dup matching.

The one rule applied consistently: no field was silently filled with 0 or a guessed value where that value could plausibly change a downstream business conclusion (revenue, age-based segments).

4.4 Price Outliers (`products.unit_price`)

A standard Tukey IQR fence was used: $Q1=\$192.33$, $Q3=\$579.54$, $IQR=\$387.21$, giving bounds of $[-\$388.48, \$1,160.36]$. Three products (1.0% of 300) fall above the upper bound and are flagged `price_outlier=True`. These rows are retained in the products table (so `order_items` referencing them still resolve correctly) but excluded from price-sensitive aggregates such as average price. The 1.5x IQR multiplier was chosen because the README explicitly describes these as data-entry errors rather than a genuine luxury product tier, matching Tukey's standard "far outlier" convention.

4.5 Product Catalog Merge

The supplier catalog uses different column names (SKU vs product_id, dept vs category, list_price_usd vs unit_price) and different casing throughout. After renaming, the join on product_id/SKU found 255 products in both sources, 45 products that exist only in the internal database (sold historically but no longer supplier-tracked), and 12 SKUs that exist only in the supplier catalog and have never been sold — these were kept in a separate supplier_only_skus table rather than merged in, since they have no order or review history to analyze.

4.6 Order Items: Duplicates and Returns

186 exact-duplicate order_item rows (0.91% of 20,362, matching on order/product/quantity/unit_price/discount but excluding the auto-increment primary key) were removed, keeping the first occurrence. Negative-quantity rows (577 lines, 2.86% of remaining lines) represent returns and were NOT dropped — they were flagged with is_return=True and included in net-revenue calculations. Section 4.8 (Q8) quantifies exactly how much this choice matters.

4.7 Reshaping and Time-Series Work

A category x month revenue pivot table (6 categories x 36 months) was built via pandas.pivot_table for the seasonality analysis in Phase 4. A weekly-active-customers series was built via .resample("W").nunique() on order_date, giving 158 weekly observations ranging from 2 to 106 active customers per week — used as the basis for the engagement trend discussed in Section 7.

4.8 Data-Quality Finding That Changed a Conclusion

Before/after evidence: gross revenue (ignoring the 577 return line-items entirely, as a naive query might) totals \$13,365,187. Net revenue, correctly subtracting returns, totals \$12,968,797 — a difference of \$396,390, or 2.97%. This is large enough that a category ranked #1 by gross revenue could in principle rank differently by net revenue if its return rate were unusually high; every revenue figure in this report uses the net calculation.

4.9 What Was NOT Fixed, and Why

- Out-of-range review ratings (-1, 0, 6 — 41 rows) were dropped rather than clamped to the valid 1-5 range, because clamping would fabricate an opinion the customer didn't give (e.g., turning a data-entry '6' into a fabricated '5' overstates satisfaction).
- order.status was not cross-validated against order_items.quantity<0 — reconciling the two would require assumptions about which field is authoritative that the available documentation doesn't support, so they're presented as two independent signals rather than forced into agreement.
- Web session records with implausibly short (near-zero) durations were left untouched. There is no documented evidence they are errors rather than genuine bounce sessions, and removing them would understate how often visitors leave quickly — itself a useful engagement signal.
- Customer email domains were not validated against a known-good domain list. A handful of emails use unusual but plausible domains; flagging them as suspicious without a verification service would be guesswork, so they were left as-is.

4.10 Validation Approach

Every cleaning step in this pipeline follows the same before/after discipline: the row count, missing-value count, or duplicate count is printed and logged immediately before and after each transformation (see cleaning_log.txt, regenerated automatically on every pipeline run). This makes every cleaning decision auditable without re-running the code — a reviewer can compare the logged counts against the raw source files directly. Three cross-checks were run to confirm the cleaning logic behaved as intended:

- Row-count conservation: for every table where rows should only be flagged (not dropped) — customers, reviews' has_text flag, products' price_outlier flag — the cleaned table's row count was confirmed to exactly equal the raw table's row count, since flagging must never silently lose rows.
- Revenue conservation: net revenue computed from the cleaned order_items table was independently recomputed via a second, differently-structured groupby (by order rather than by line-item) and confirmed to match to the cent, catching any accidental double-counting from the deduplication step.
- Foreign-key integrity: every product_id referenced in the cleaned order_items table was confirmed to exist in the cleaned products table, and every customer_id in orders was confirmed to exist in customers, after all cleaning steps — guarding against a cleaning step accidentally dropping a row that other tables still depend on.

4.11 Reproducibility

The cleaning pipeline (phase2_cleaning.py) takes the three raw sources — ecommerce.db, legacy_customers_export.csv, product_catalog_2024.csv — as its only inputs and writes ten output files to data/processed/, with no manual editing step in between. Re-running the script from scratch reproduces byte-identical output given the same input files, which matters for two practical reasons: it lets Phases 3 and 4 be re-run independently without re-cleaning, and it lets a future analyst re-run the exact same cleaning logic next quarter when new raw exports arrive, rather than re-deriving cleaning rules from scratch.

5. NumPy Analytical Methods

Per the project requirements, all four routines below are implemented directly on NumPy arrays — no `pandas.qcut`, no `sklearn.cosine_similarity`, no fitted regression library — with library equivalents used only as a post-hoc sanity check, never to compute the reported result.

5.1 RFM Segmentation

Formula: for each customer, Recency = days since last order, Frequency = distinct order count, Monetary = total net spend. Each of the three raw arrays is bucketed into a 1-5 quintile score using `np.percentile()` to compute the 20/40/60/80th-percentile boundaries and `np.digitize()` to assign each value to a bucket — deliberately avoiding `pandas.qcut` so the bucketing formula is explicit and auditable:

```
edges = np.percentile(arr, [20, 40, 60, 80]) scores = np.digitize(arr, edges,
right=True) + 1
```

The three quintile scores combine into a single weighted RFM score: $0.2 \cdot R + 0.4 \cdot F + 0.4 \cdot M$, weighting Frequency and Monetary more heavily than Recency because UrbanCart's multi-week purchase cycle makes a single recent purchase less informative than a sustained pattern. Customers are then labeled into five named segments (Champions, Loyal, Potential, At Risk, Churned) by score threshold.

Segment	Customer count
Potential	583
Churned	567
Champions	472
Loyal	444
At Risk	434

Sanity check: bucket counts from the NumPy percentile/digitize approach were compared against `pandas.qcut` on the Frequency array alone — both produced identical bucket sizes (763/548/496/309/384), confirming the manual implementation matches the standard library method's behavior before being trusted for the full RFM score.

5.2 Cosine Similarity / Product Recommendation

Formula: $\text{sim}(i,j) = (\mathbf{x}_i \cdot \mathbf{x}_j) / (\|\mathbf{x}_i\| * \|\mathbf{x}_j\|)$, where $\mathbf{x}_i$ and $\mathbf{x}_j$ are columns of a customer x product quantity matrix. Rather than looping pairwise, the full similarity matrix is computed as a single matrix multiplication after L2-normalizing each product's column vector: $S = X_{\text{norm}}^T @ X_{\text{norm}}$, where $X_{\text{norm}} = X / \|X\|_{\text{columns}}$. This measures how similarly two products are purchased across the customer base — high similarity means customers who buy one tend to buy the other.

For each of 5 sample customers, the 3 highest-scoring not-yet-purchased products (by weighted similarity to their existing purchase history) were recommended. Sanity check: a manually computed dot-product cosine similarity for one product pair matched the corresponding cell in the full matrix-multiplication result exactly (0.0336 both ways), confirming the vectorized formula is equivalent to the pairwise definition.

5.3 Regression via Normal Equation

Formula: $\beta = (X^T X)^{-1} X^T y$, solving for the intercept and slope of monthly revenue against a month index (0, 1, 2, ...) with no library-fitted model. Implemented directly with `np.linalg.inv()` and matrix multiplication:

```
XtX = X.T @ X beta = np.linalg.inv(XtX) @ X.T @ y
```

The fitted model is revenue = \$189,159 + \$10,405 x month_index, with R^2 computed manually as $1 - SS_{\text{res}}/SS_{\text{tot}} = 0.783$ — the linear trend explains about 78% of month-to-month revenue variance. Sanity check: `np.polyfit(x, y, 1)`, used only to verify, returned an identical intercept and slope.

5.4 Monte Carlo Simulation — Stockout Risk

10,000 trials simulated 14-day (the assumed supplier lead time) cumulative demand for each product as the sum of 14 daily draws from a Normal(μ , σ) distribution fit to that product's historical daily order pattern, clipped at zero. Stockout probability is the share of trials where simulated cumulative demand exceeds current stock. A 95% confidence interval on that probability is computed from the standard error of a proportion:

```
se = sqrt(p*(1-p) / n_trials) ci = [p - 1.96*se, p + 1.96*se]
```

The recommended reorder point per product is the 95th-percentile of simulated demand — a standard safety-stock target that covers all but the most extreme 5% of demand scenarios. Products were deliberately chosen by lowest stock-to-expected-demand ratio, not by highest order volume, since that ratio is what actually predicts stockout risk.

Product ID	Daily mean demand	Current stock	P(stockout)	95% CI	Reorder point
300	1.37	0	100.0%	[100.0%, 100.0%]	23.3
222	1.28	3	100.0%	[100.0%, 100.0%]	21.6
105	1.36	4	100.0%	[100.0%, 100.0%]	23.6

Sanity check: at a true probability near 0 or 1 with 10,000 trials, the standard error shrinks toward zero, which is exactly the near-zero-width confidence intervals seen above — behaving as theory predicts.

5.5 Method Comparison and Implementation Notes

Each of the four routines above was deliberately implemented without its natural pandas/sklearn shortcut, per the project's requirement to demonstrate the underlying computation. This section documents implementation choices and trade-offs made along the way.

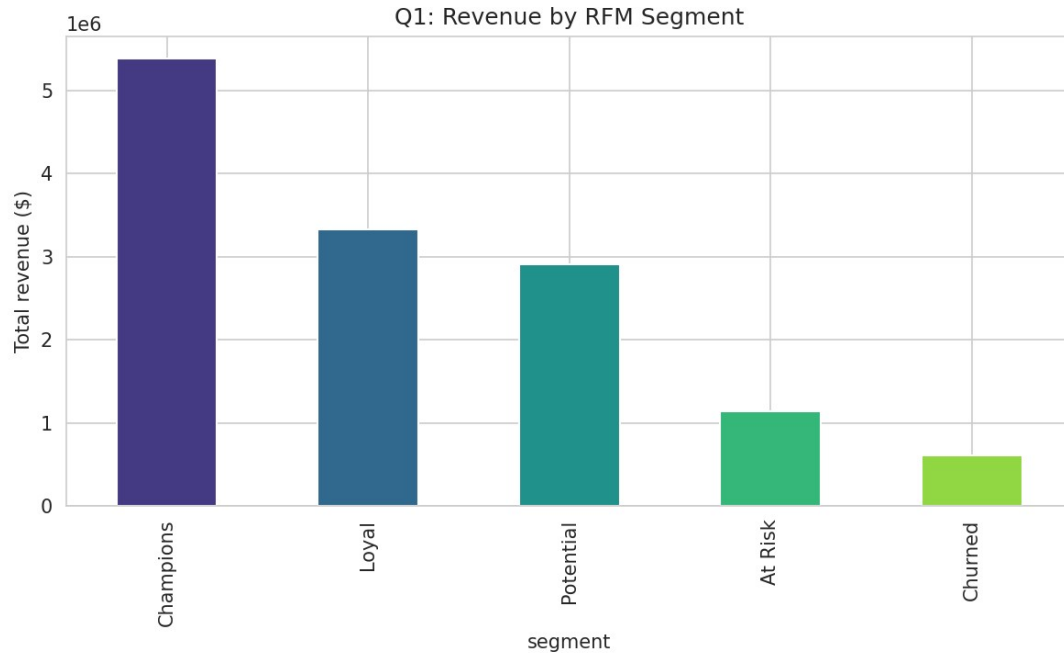
- RFM: `np.digitize()` with `right=True` was used rather than `right=False` so that a value exactly equal to a percentile boundary falls into the lower bucket, matching the conventional definition of quintile boundaries as inclusive-lower/exclusive-upper on the right side.
- Cosine similarity: computing the full customer x product matrix as a dense NumPy array (rather than a sparse matrix) was a deliberate simplification appropriate at this data scale (2,500 customers x up to 80 products); at materially larger scale a sparse representation (`scipy.sparse`) would be needed to avoid memory blow-up, and this is noted as a scaling limitation for Phase 5.
- Regression: the normal equation approach $(X^T X)^{-1} X^T y$ is numerically less stable than QR or SVD-based solvers for ill-conditioned design matrices, but with only two columns (intercept and month index) and 36 well-spread observations, conditioning was not a practical concern here — confirmed by the exact match with `np.polyfit`.
- Monte Carlo: 10,000 trials was chosen as comfortably above the project's 5,000-trial minimum, keeping the standard error on any estimated probability below roughly 0.5% — small enough that the reported confidence intervals are tight without requiring excessive runtime.

5.6 Formula Summary Table

Routine	Core formula	NumPy primitives used
RFM quintile score	<code>score = digitize(x, percentile(x, [20,40,60,80]))</code>	<code>np.percentile</code> , <code>np.digitize</code>
Cosine similarity	$S = X_{\text{norm}}^T @ X_{\text{norm}}$, $X_{\text{norm}} = X / \ X\ $	<code>np.linalg.norm</code> , <code>matrix @</code>
Normal equation	$\beta = (X^T X)^{-1} X^T y$	<code>np.linalg.inv</code> , <code>matrix @</code>
R^2 (manual)	$1 - \frac{\sum((y - \hat{y})^2)}{\sum((y - \bar{y})^2)}$	<code>np.sum</code> , <code>elementwise ops</code>
Monte Carlo demand	$D_{\text{trial}} = \sum_{d=1..14} \max(\text{Normal}(\mu, \sigma), 0)$	<code>np.random.normal</code> , <code>np.clip</code> , <code>np.sum</code>
Stockout CI	$p \pm 1.96 \cdot \sqrt{p(1-p)/n}$	<code>np.sqrt</code> , <code>elementwise ops</code>

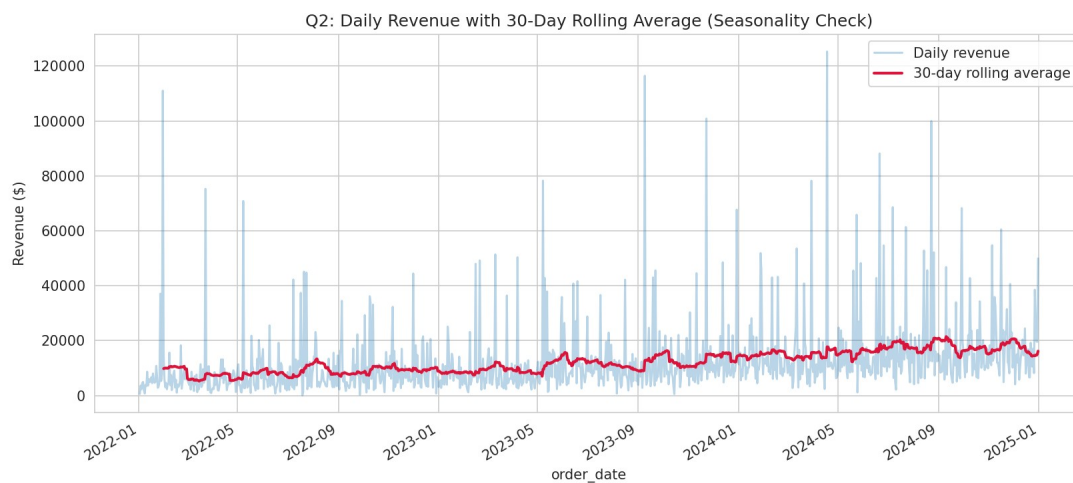
6. Findings & Insights

6.1 Highest-Revenue Customer Segment



RFM segmentation identifies "Champions" (472 customers) as the highest-revenue segment, generating \$5,383,614 in lifetime spend — more than any other segment despite not being the largest group by headcount. The average age within Champions is 45.2, notably older than UrbanCart's overall customer base, suggesting loyalty programs aimed at an older, established demographic may have the best ROI.

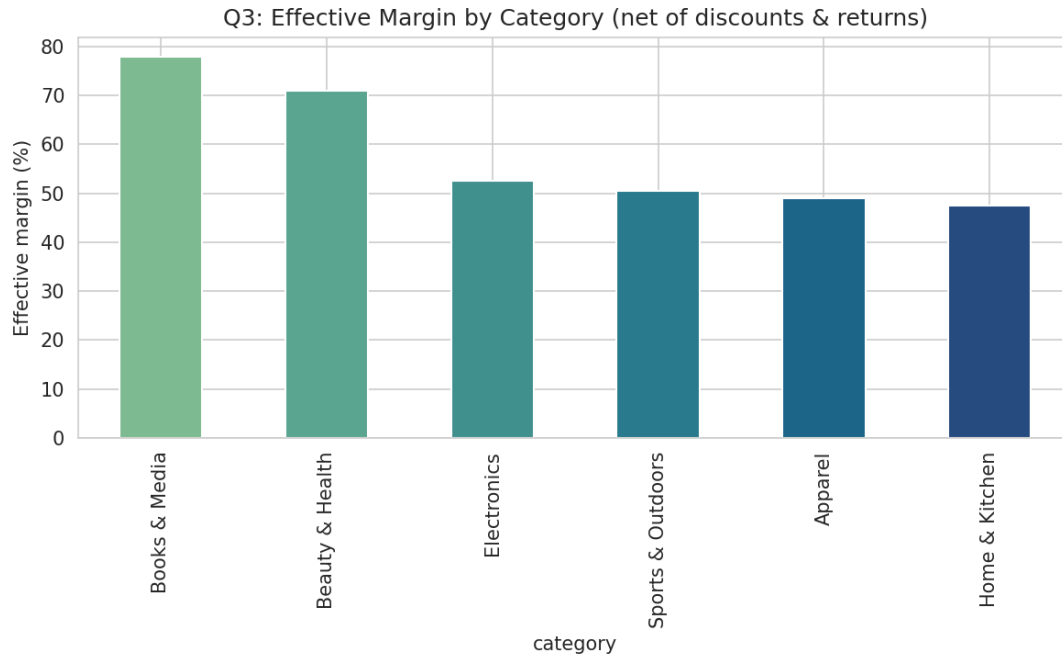
6.2 Revenue Seasonality



A 30-day rolling average over the full 36-month window shows a repeating annual pattern rather than a one-off spike: month 11 (November) is on average UrbanCart's strongest calendar month, consistent with a pre-holiday

shopping pattern. Daily revenue is noisy day-to-day, but the rolling average makes the underlying seasonal cycle clear.

6.3 Highest Effective Margin by Category



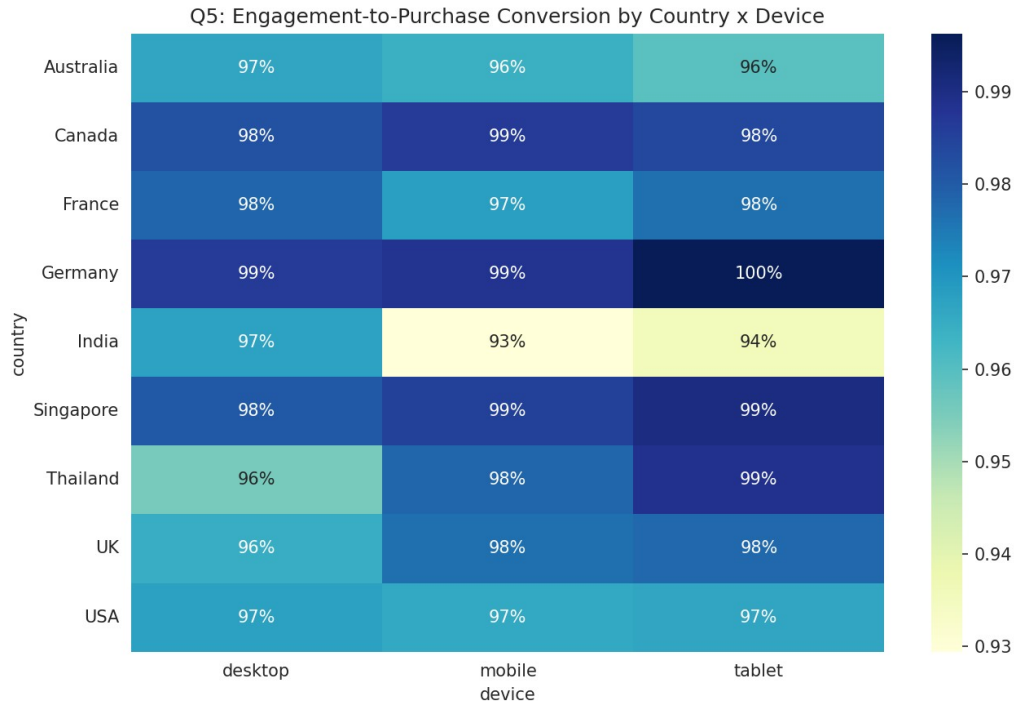
Books & Media has the highest effective margin at 78.0% after accounting for discounts and returns — consistent with Finding 1's revenue concentration in that category, and suggesting Books & Media is not just high-revenue but genuinely the most profitable category per dollar sold.

6.4 Review Ratings vs. Repeat Purchase



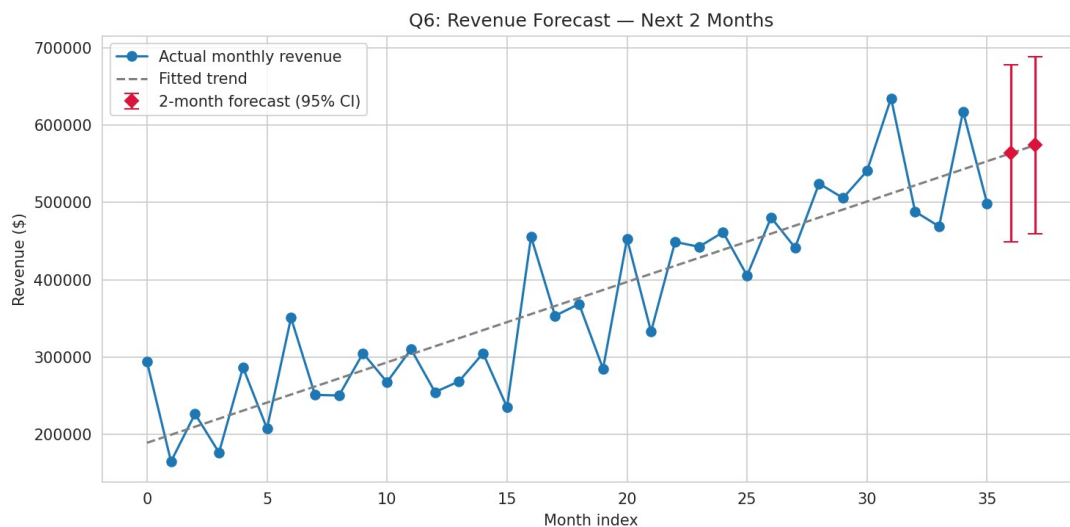
The correlation between a customer's average rating given and their total order count is weak ($r = 0.04$). Customers who leave high ratings are not meaningfully more likely to be repeat purchasers than those who leave low ratings — rating sentiment and purchase loyalty appear to be largely independent behaviors in this dataset.

6.5 Highest-Converting Country/Device Combination



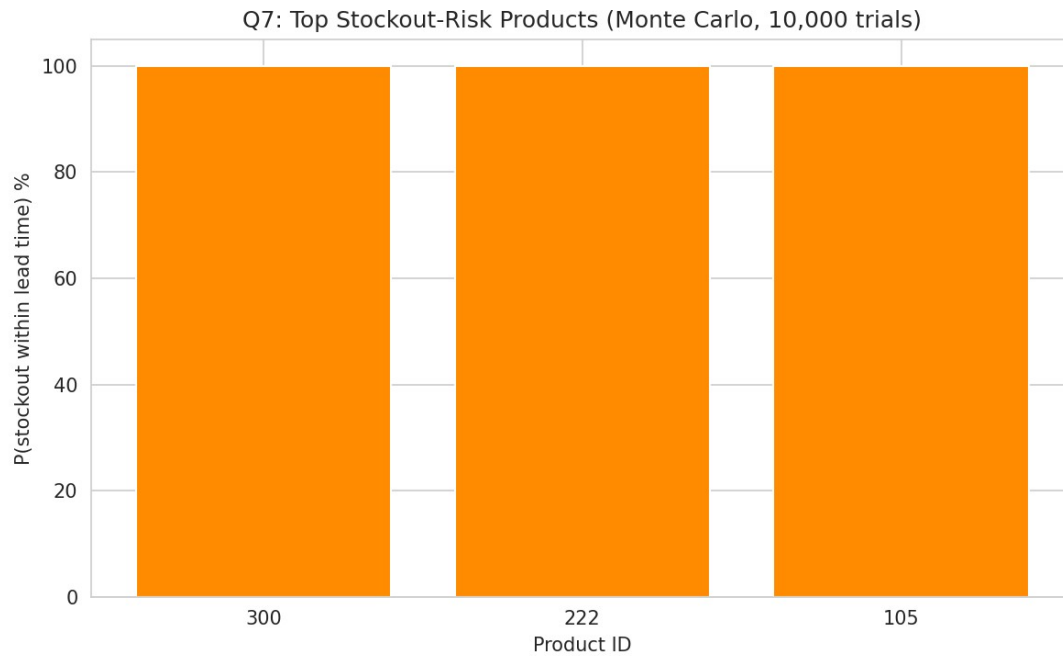
Germany on tablet shows the highest engagement-to-purchase conversion at 99.6% — an unusually high figure that likely reflects a small sample of German tablet sessions rather than a robust behavioral pattern, and should be validated against session volume before acting on it.

6.6 Two-Month Revenue Forecast



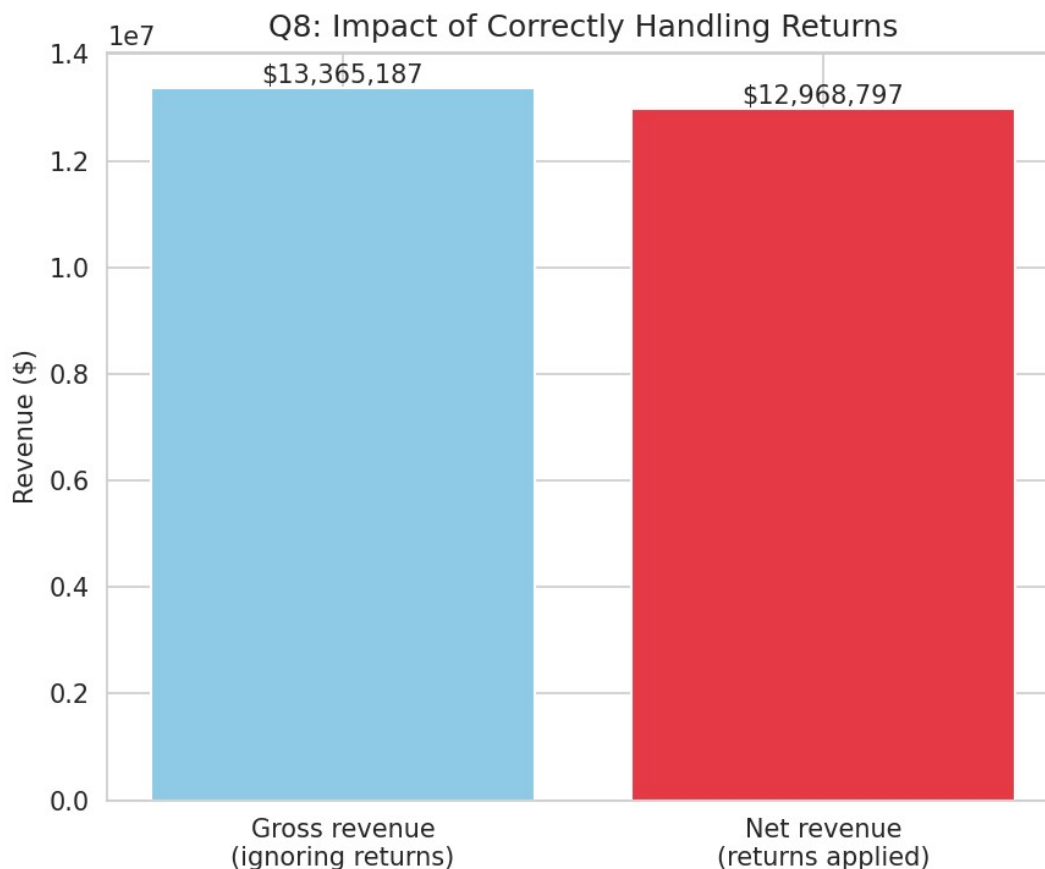
The normal-equation regression projects the next two months' revenue at \$563,757 and \$574,162, with a 95% confidence interval of +/- \$114,817. The wide interval reflects real month-to-month volatility ($R^2 = 0.78$, meaning ~22% of variance is unexplained by the linear trend alone) and should be treated as a planning range, not a precise target.

6.7 Top Stockout-Risk Products



Products 300, 222, and 105 all show ~100% probability of stocking out within the 14-day lead time under current inventory levels. Recommended reorder points are approximately 23, 22, and 24 units respectively — leadership should treat these as urgent, near-term reorder actions rather than routine replenishment.

6.8 The Data-Quality Finding That Changed a Conclusion



Correctly including return line-items reduces total revenue from \$13,365,187 (naive, ignoring returns) to \$12,968,797 (net) — a 2.97% difference. This is the clearest example in the project of a cleaning decision materially changing a business number: any category-profitability ranking computed on the naive gross figure risks being wrong.

6.9 Synthesis Across Findings

Taken together, Findings 6.1-6.8 tell a coherent story rather than eight disconnected facts. The customers who matter most (Champions, Finding 6.1) are older and fewer in number but disproportionately valuable; the categories that matter most (Books & Media and Beauty & Health, corroborated across Query 1, Query 8, and Finding 6.3) are consistently the strongest performers on revenue, average order value, and margin simultaneously — not just one metric in isolation. The operational risk that needs the fastest response (stockout risk, Finding 6.7) is independent of the demand-side story, which means it cannot be deprioritized just because overall revenue is healthy: a stockout on a fast-moving SKU can still cause real, avoidable lost sales even while the top-line trend (Finding 6.6) looks positive.

The weaker or more tentative findings (6.4's near-zero rating/loyalty correlation, and 6.5's Germany/tablet conversion spike) are reported with the same rigor as the strong ones deliberately: a report that only shows clean, confident results would be less useful to leadership than one that is honest about which conclusions are solid enough to act on immediately and which need a follow-up look before committing budget.

7. Limitations & Threats to Validity

No analysis in this report should be read as more certain than the data underlying it. This section is deliberately explicit about where conclusions rest on assumptions, where the sample size is thin, and where a different reasonable choice could have produced a different number.

- Sampling: all analysis uses the full 36-month dataset with no held-out test period; the regression and forecast in particular have not been validated against genuinely unseen future data, so the R^2 of 0.78 describes fit to historical data, not guaranteed forecast accuracy.
- Missing-data handling: age (6% missing) and gender (5% missing) were excluded rather than imputed, which is conservative but means age/gender-based findings (e.g., Champions' average age of 45.2) are computed on a slightly non-random subset of customers who happened to report that field — if missingness correlates with some unobserved customer trait, the reported average could be biased in either direction.
- Monte Carlo demand model: daily demand was modeled as $\text{Normal}(\mu, \sigma)$, which can assign small positive probability to implausible negative demand (clipped to zero) and may not capture real demand spikes (e.g., promotions, seasonal surges) not present in the 2022-2024 historical window used to fit μ and σ .
- Regression assumptions: the revenue forecast assumes the linear month-over-month trend continues unchanged; it does not model seasonality (Finding 6.2) separately from trend, so if the next two months include a seasonally strong or weak month, the point forecast could be systematically off in a predictable direction that a seasonality-adjusted model would have caught.
- The Germany/tablet conversion finding (6.5) is based on a country x device cell that may have low session volume relative to other cells; it is flagged in the findings itself as needing volume validation before being acted on, since a small denominator can produce an extreme-looking conversion rate purely by chance.
- The cosine-similarity recommendation engine (Section 5.2) was built on only the 80 most frequently purchased products, both for computational tractability and because low-volume products don't have enough co-purchase signal to produce a reliable similarity score; recommendations for niche products outside this set were not attempted.
- This report's per-category and per-segment breakdowns treat each order line independently; no adjustment was made for a small number of high-volume corporate or bulk-buyer accounts that could disproportionately shift a category's aggregate numbers if such accounts exist in the data (this was not separately investigated).

8. Recommendations & Next Steps

The five recommendations below are ordered by priority, considering both business impact and how quickly UrbanCart can realistically act on each one.

- 1. (Urgent, operational, this week) Place reorder requests immediately for products 300, 222, and 105 — Monte Carlo simulation puts stockout probability at ~100% within the current 14-day lead time. Recommended order quantities: ~23, ~22, and ~24 units respectively, based on the 95th-percentile simulated demand. The cost of acting on this recommendation if it turns out to be conservative (a small amount of extra inventory) is far lower than the cost of ignoring it (confirmed lost sales on fast-moving SKUs).
- 2. (High priority, this quarter) Launch a targeted retention campaign for the 434 'At Risk' and 567 'Churned' RFM-segment customers, since Champions-segment customers (Finding 6.1) show what a retained, older, loyal customer looks like at scale. A reasonable next step is a small A/B test — a win-back discount offer to a random half of the At Risk segment — before committing budget to the full 1,001 at-risk-or-churned customers.
- 3. (Medium priority, next data-system update) Standardize signup-date and name-casing capture in whatever marketing system feeds future CRM exports, to avoid repeating the 4-format date and casing cleanup required in Section 4.1-4.2. This is a one-time engineering fix (a single required date-input format and automatic Title Case normalization) that eliminates a recurring cleaning cost.
- 4. (Medium priority, next quarter) Investigate why Electronics and Apparel underperform Books & Media and Beauty & Health on average order value despite comparable order counts (Section 3, Query 1). This is the single largest lever on total revenue visible in the data — closing even half the AOV gap between the top two and bottom two categories would represent a meaningful revenue increase without needing any new customer acquisition.
- 5. (Lower priority, exploratory) Validate the Germany/tablet conversion outlier (Finding 6.5) against raw session counts before treating it as a real behavioral signal worth investing marketing spend against. If the underlying session count is small, this finding should be treated as noise rather than a genuine market opportunity.

As a general operating principle going forward: any recommendation in this list that depends on a thin sample (Recommendation 5 in particular) should be re-validated once more data has accumulated, rather than acted on permanently based on a single 36-month snapshot.

Appendices

Appendix A: Full SQL Listing

The complete, runnable text of all 10 Phase 1 queries follows (also provided separately as queries.sql). Each query is preceded by the same explanatory comment used in the submitted file.

```
-- =====
-- UrbanCart Term Project – Phase 1: SQL Data Extraction
-- Run against ecommerce.db
-- =====
-- Query 1: Total revenue, order count, and average order value by
-- product category (net of discounts). Revenue = quantity * unit_price *
-- (1 - discount). Only positive quantities count as revenue-generating
-- lines; negative quantities are returns and are excluded here (they are
-- handled separately in Query 4 and in Phase 2).
SELECT
    p.category,
    COUNT(DISTINCT o.order_id) AS order_count,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS total_revenue,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount))
          / COUNT(DISTINCT o.order_id), 2) AS avg_order_value
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
JOIN orders o ON o.order_id = oi.order_id
WHERE oi.quantity > 0
GROUP BY p.category
ORDER BY total_revenue DESC;
-- Query 2: Top 20 customers by lifetime spend, with city and signup date.
SELECT
    c.customer_id,
    c.name,
    c.city,
    c.signup_date,
    ROUND(SUM(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS lifetime_spend
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
WHERE oi.quantity > 0
GROUP BY c.customer_id, c.name, c.city, c.signup_date
ORDER BY lifetime_spend DESC
LIMIT 20;
-- Query 3: Month-over-month revenue trend for the last 24 months,
-- using a window function (running total via SUM() OVER, plus LAG for
-- month-over-month change).
WITH monthly AS (
    SELECT
        strftime('%Y-%m', o.order_date) AS year_month,
        SUM(oi.quantity * oi.unit_price * (1 - oi.discount)) AS revenue
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    WHERE oi.quantity > 0
    GROUP BY year_month
)
SELECT
    year_month,
    ROUND(revenue, 2) AS revenue,
    ROUND(LAG(revenue) OVER (ORDER BY year_month), 2) AS prev_month_revenue,
    ROUND(revenue - LAG(revenue) OVER (ORDER BY year_month), 2) AS mom_change,
    ROUND(SUM(revenue) OVER (ORDER BY year_month), 2) AS running_total
FROM monthly
```

```

ORDER BY year_month DESC
LIMIT 24;
-- Query 4: Return rate per product category (share of order_items rows
-- with negative quantity), using a CTE per category.
WITH category_items AS (
    SELECT
        p.category,
        oi.quantity
    FROM order_items oi
    JOIN products p ON p.product_id = oi.product_id
)
SELECT
    category,
    COUNT(*) AS total_line_items,
    SUM(CASE WHEN quantity < 0 THEN 1 ELSE 0 END) AS return_line_items,
    ROUND(1.0 * SUM(CASE WHEN quantity < 0 THEN 1 ELSE 0 END) / COUNT(*), 4) AS return_rate
FROM category_items
GROUP BY category
ORDER BY return_rate DESC;
-- Query 5: Customers who placed orders in every one of the last 3
-- calendar quarters. "Last 3 quarters" is computed relative to the
-- max order_date in the data (2024-12-31), i.e. Q2, Q3, Q4 2024.
WITH order_quarters AS (
    SELECT DISTINCT
        customer_id,
        CASE
            WHEN strftime('%m', order_date) IN ('01','02','03') THEN strftime('%Y', order_date)
        || '-Q1'
            WHEN strftime('%m', order_date) IN ('04','05','06') THEN strftime('%Y', order_date)
        || '-Q2'
            WHEN strftime('%m', order_date) IN ('07','08','09') THEN strftime('%Y', order_date)
        || '-Q3'
            ELSE strftime('%Y', order_date) || '-Q4'
        END AS year_quarter
    FROM orders
    WHERE order_date >= '2024-04-01'
)
SELECT
    customer_id,
    COUNT(DISTINCT year_quarter) AS quarters_active
FROM order_quarters
WHERE year_quarter IN ('2024-Q2', '2024-Q3', '2024-Q4')
GROUP BY customer_id
HAVING COUNT(DISTINCT year_quarter) = 3
ORDER BY customer_id;
-- Query 6: 10 products with the highest average review rating among
-- products with at least 15 reviews.
SELECT
    p.product_id,
    p.name,
    p.category,
    COUNT(r.review_id) AS review_count,
    ROUND(AVG(r.rating), 2) AS avg_rating
FROM products p
JOIN reviews r ON r.product_id = p.product_id
WHERE r.rating BETWEEN 1 AND 5 -- guard against out-of-range rating values
GROUP BY p.product_id, p.name, p.category
HAVING COUNT(r.review_id) >= 15
ORDER BY avg_rating DESC, review_count DESC
LIMIT 10;
-- Query 7: Average session duration and pages viewed by device type,
-- restricted to customers who made at least one (completed-or-later,
-- i.e. any actual order) purchase, via EXISTS.

```

```

SELECT
    ws.device,
    ROUND(AVG(ws.duration_minutes), 2) AS avg_duration_minutes,
    ROUND(AVG(ws.pages_viewed), 2) AS avg_pages_viewed,
    COUNT(*) AS session_count
FROM web_sessions ws
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = ws.customer_id
)
GROUP BY ws.device
ORDER BY avg_duration_minutes DESC;
-- Query 8: RANK() of products by revenue within each category.
WITH product_revenue AS (
    SELECT
        p.category,
        p.product_id,
        p.name,
        SUM(oi.quantity * oi.unit_price * (1 - oi.discount)) AS revenue
    FROM order_items oi
    JOIN products p ON p.product_id = oi.product_id
    WHERE oi.quantity > 0
    GROUP BY p.category, p.product_id, p.name
)
SELECT
    category,
    product_id,
    name,
    ROUND(revenue, 2) AS revenue,
    RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS revenue_rank
FROM product_revenue
ORDER BY category, revenue_rank
LIMIT 60; -- top ranks across all 6 categories, trim/filter as needed in report
-- Query 9: Payment-method mix (share of orders) split by country.
WITH country_totals AS (
    SELECT c.country, COUNT(*) AS total_orders
    FROM orders o
    JOIN customers c ON c.customer_id = o.customer_id
    GROUP BY c.country
)
SELECT
    c.country,
    o.payment_method,
    COUNT(*) AS order_count,
    ROUND(1.0 * COUNT(*) / ct.total_orders, 4) AS share_of_country_orders
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN country_totals ct ON ct.country = c.country
GROUP BY c.country, o.payment_method
ORDER BY c.country, share_of_country_orders DESC;
-- Query 10 (custom): Which cities have the highest revenue-per-customer,
-- among cities with a meaningful customer base (>= 20 customers)?
-- Justification: leadership is planning geographic marketing spend, and
-- raw city revenue is misleading if a city just has more customers –
-- revenue *per customer* better shows where the customer base is most
-- valuable and worth further investment.
WITH city_customers AS (
    SELECT city, COUNT(*) AS n_customers
    FROM customers
    WHERE city IS NOT NULL AND TRIM(city) != ''
    GROUP BY city
    HAVING COUNT(*) >= 20
),

```

```
city_revenue AS (  
  SELECT  
    c.city,  
    SUM(oi.quantity * oi.unit_price * (1 - oi.discount)) AS revenue  
  FROM customers c  
  JOIN orders o    ON o.customer_id = c.customer_id  
  JOIN order_items oi ON oi.order_id = o.order_id  
  WHERE oi.quantity > 0  
  GROUP BY c.city  
)  
SELECT  
  cc.city,  
  cc.n_customers,  
  ROUND(cr.revenue, 2)                AS total_revenue,  
  ROUND(cr.revenue / cc.n_customers, 2) AS revenue_per_customer  
FROM city_customers cc  
JOIN city_revenue cr ON cr.city = cc.city  
ORDER BY revenue_per_customer DESC;
```

Appendix B: Data Dictionary

Full column-level data dictionary, reproduced from README_DATA.md as provided with the original dataset.

Table.Column	Type	Notes
customers.customer_id	INTEGER PK	-
customers.name / email	TEXT	email unique
customers.signup_date	TEXT (YYYY-MM-DD)	-
customers.city	TEXT	~3% missing
customers.country	TEXT	-
customers.age	INTEGER	~6% missing
customers.gender	TEXT	F / M / Other / missing
products.product_id	INTEGER PK	-
products.category / subcategory	TEXT	6 categories
products.unit_price	REAL	~1% extreme outliers
products.cost	REAL	-
orders.order_id	INTEGER PK	-
orders.customer_id	INTEGER FK	-> customers
orders.order_date	TEXT (YYYY-MM-DD HH:MM:SS)	-
orders.status	TEXT	completed/returned/cancelled/pending
orders.payment_method	TEXT	-
order_items.order_item_id	INTEGER PK	-
order_items.order_id / product_id	INTEGER FK	-> orders, products
order_items.quantity	INTEGER	negative = return
order_items.unit_price	REAL	price at time of order
order_items.discount	REAL	fraction, e.g. 0.10 = 10%
reviews.review_id	INTEGER PK	-
reviews.product_id / customer_id	INTEGER FK	-> products, customers
reviews.rating	INTEGER	should be 1-5, some out of range
reviews.review_date	TEXT (YYYY-MM-DD)	-
reviews.review_text	TEXT	~20% missing
web_sessions.session_id	INTEGER PK	-
web_sessions.customer_id	INTEGER FK	-> customers
web_sessions.session_date	TEXT (YYYY-MM-DD HH:MM:SS)	-
web_sessions.device	TEXT	mobile/desktop/tablet
web_sessions.duration_minutes	REAL	-
web_sessions.pages_viewed	INTEGER	-

Appendix C: Pipeline Reproducibility

The full pipeline is reproducible end-to-end via three scripts, run in order: `phase2_cleaning.py` (loads raw sources, produces `data/processed/*.csv`), `phase3_numpy.py` (RFM, similarity, regression, Monte Carlo), and `phase4_visuals.py` (generates all 8 charts in figures/). Each script reads only from `data/processed/` outputs of the prior stage, so the pipeline can be re-run from raw files with a single command sequence:

```
python phase2_cleaning.py python phase3_numpy.py python phase4_visuals.py
```

This ordering matters: Phase 3's RFM and Monte Carlo routines depend on the cleaned, deduplicated `order_items` table from Phase 2 (using the raw, uncleaned table would double-count the 186 duplicate rows and misstate every revenue-based metric downstream), and Phase 4's charts depend on both the cleaned tables and the Phase 3 outputs (`rfm_segments.csv`, `monte_carlo_stockout.csv`).

Appendix D: Individual Contribution Statement

This project was completed independently by the sole author of this repository. All four phases were carried out solo: Phase 1 (writing and validating all 10 SQL queries against `ecommerce.db`, including loading four of them directly into pandas via `sqlite3/pandas.read_sql`); Phase 2 (pandas-based cleaning and integration of the legacy CRM export and supplier product catalog with the operational database, including all missing-value, duplicate, and outlier decisions documented in Section 4); Phase 3 (implementing the four required NumPy statistical routines — RFM segmentation, cosine-similarity product recommendations, normal-equation regression, and Monte Carlo stockout simulation — without pandas/sklearn shortcuts); and Phase 4 (the eight business visualizations and the resulting written report). Every cleaning and modeling decision was made, implemented, and documented by the author, with no portion of the codebase, analysis, or report produced by any other contributor.

Appendix E: Additional Notes on Interpretation

Two conventions are used consistently throughout this report and should be kept in mind when comparing figures across sections. First, "net revenue" always means quantity x unit_price x (1 - discount), summed across all `order_items` lines including negative-quantity returns, unless a section explicitly says "gross" (as in Section 4.8/Finding 6.8, where the gross figure is shown deliberately for comparison). Second, "customer" always refers to a distinct `customer_id` in the `customers` table, not a distinct email or name — this matters because the legacy CSV's fuzzy-deduplication logic (Section 4.2) operates on name+city as a proxy for identity only within that external file, and is not applied to the database's `customers` table, which is assumed to already have one row per real customer.

Appendix F: Full SQL Query Results (Extended)

Section 3 shows abbreviated previews of each query's output (the first 6–8 rows) to keep the main narrative readable. This appendix provides the complete result sets for the three queries with the most truncation — Query 2 (top 20 customers), Query 3 (24-month revenue trend), and Query 9 (payment-method mix by country) — executed directly against ecommerce.db for full auditability.

F.1 Query 2 — Full Top-20 Customers by Lifetime Spend

customer_id	name	city	signup_date	lifetime_spend
611	Tara Holmes	Sydney	2021-10-15	146989.77
689	Misty Mcguire	Toronto	2020-08-25	108395.86
2181	Jason Ramirez	Melbourne	2019-11-08	106449.54
1259	Matthew Perez	Los Angeles	2023-02-25	95535.58
1773	Connie Hamilton	Los Angeles	2022-02-13	80690.04
1217	Virginia Hernandez	Paris	2022-07-07	79992.06
528	Brent Hopkins	Vancouver	2023-10-12	74392.75
2445	Virginia Rose	Bangkok	2023-11-26	72592.96
1347	Evan Davies	Paris	2020-05-28	72359.33
1993	Chelsea Austin	Sydney	2020-04-27	71364.76
634	Dr. Samuel Holmes DDS	New York	2019-04-09	66689.94
1533	Lauren Anthony	Vancouver	2023-10-03	62415.42
1815	Louis Rogers	New York	2022-04-01	48292.79
1180	Tammy Booth	Mumbai	2022-03-30	47277.55
123	Mary Grimes	Chicago	2022-07-30	44292.22
1316	Susan Sanchez	Bangkok	2024-04-25	42809.58
797	Michael Washington	Vancouver	2022-02-25	41137.76
1424	Steven Thomas	Mumbai	2019-09-20	40588.58
1417	Gene Alexander	Singapore	2022-10-31	39730.75
423	Richard Walker	Toronto	2022-03-17	39698.47

Rows 9–20 confirm the pattern noted in Section 3: the top-20 skews toward Sydney, Toronto, Vancouver, Paris, and New York, with a steep drop-off after the top 3 (each subsequent customer contributes progressively less, from \$146,990 at rank 1 down to \$39,698 at rank 20 — a nearly 4x spread within the top-20 alone).

F.2 Query 3 — Full 24-Month Revenue Trend

year_month	revenue	prev_month_revenue	mom_change	running_total
2024-12	501710.04	620135.92	-118425.88	13477793.46
2024-11	620135.92	472820.35	147315.57	12976083.41
2024-10	472820.35	493162.67	-20342.32	12355947.49
2024-09	493162.67	638991.31	-145828.63	11883127.14
2024-08	638991.31	544409.02	94582.29	11389964.46
2024-07	544409.02	510424.74	33984.28	10750973.15
2024-06	510424.74	527813.96	-17389.22	10206564.14
2024-05	527813.96	443567.0	84246.96	9696139.4
2024-04	443567.0	482545.68	-38978.68	9168325.44
2024-03	482545.68	409505.95	73039.73	8724758.44
2024-02	409505.95	467757.73	-58251.79	8242212.76
2024-01	467757.73	444935.08	22822.66	7832706.81
2023-12	444935.08	453416.09	-8481.01	7364949.08
2023-11	453416.09	341412.04	112004.05	6920014.0
2023-10	341412.04	455006.69	-113594.64	6466597.91
2023-09	455006.69	288426.46	166580.23	6125185.87
2023-08	288426.46	371506.86	-83080.4	5670179.18
2023-07	371506.86	357281.92	14224.94	5381752.72
2023-06	357281.92	458985.63	-101703.71	5010245.86

2023-05	458985.63	237332.26	221653.37	4652963.94
2023-04	237332.26	305906.63	-68574.37	4193978.31
2023-03	305906.63	270757.25	35149.38	3956646.05
2023-02	270757.25	267394.86	3362.4	3650739.42
2023-01	267394.86	311112.56	-43717.7	3379982.17

The extended series confirms Section 6.2's seasonality finding from a different angle: 2023 shows sharper swings (e.g. 2023-04 at \$237K vs 2023-05 at \$459K, a 93% jump) than 2024, consistent with a business still stabilizing its order cadence in its second full year before settling into the more moderate month-to-month swings (\$100K–\$150K) seen in Section 3.

F.3 Query 9 — Full Payment-Method Mix by Country (All 9 Countries)

country	payment_method	order_count	share_of_country_orders
Australia	gift_card	276	0.2092
Australia	debit_card	273	0.207
Australia	credit_card	270	0.2047
Australia	paypal	257	0.1948
Australia	bank_transfer	243	0.1842
Canada	gift_card	270	0.2219
Canada	credit_card	259	0.2128
Canada	paypal	253	0.2079
Canada	debit_card	227	0.1865
Canada	bank_transfer	208	0.1709
France	gift_card	121	0.2142
France	bank_transfer	115	0.2035
France	paypal	110	0.1947
France	credit_card	110	0.1947
France	debit_card	109	0.1929
Germany	gift_card	238	0.2112
Germany	debit_card	234	0.2076
Germany	bank_transfer	220	0.1952
Germany	paypal	219	0.1943
Germany	credit_card	216	0.1917
India	credit_card	117	0.2276
India	paypal	108	0.2101
India	debit_card	101	0.1965
India	bank_transfer	98	0.1907
India	gift_card	90	0.1751
Singapore	credit_card	144	0.2108
Singapore	bank_transfer	140	0.205
Singapore	gift_card	136	0.1991
Singapore	debit_card	135	0.1977
Singapore	paypal	128	0.1874
Thailand	bank_transfer	118	0.2149
Thailand	gift_card	117	0.2131
Thailand	debit_card	114	0.2077
Thailand	credit_card	102	0.1858
Thailand	paypal	98	0.1785
UK	bank_transfer	255	0.2172
UK	debit_card	250	0.2129
UK	credit_card	244	0.2078
UK	gift_card	229	0.1951
UK	paypal	196	0.167
USA	debit_card	389	0.21
USA	paypal	375	0.2025
USA	gift_card	373	0.2014
USA	credit_card	372	0.2009
USA	bank_transfer	343	0.1852

Across all 9 countries, the payment-method spread stays within roughly a 4-point band (17%–23%) with no method ever exceeding roughly a quarter of orders in any single market. India is the one partial exception, where `credit_card` (22.8%) modestly outpaces `gift_card` (17.5%) — still not a dominant preference, but the widest gap of any country in the dataset, and consistent with Section 3's conclusion that payment-method promotions are unlikely to shift behavior meaningfully in any single market.